

AutoCamTracker V1 平行開發版本比較分析報告

日期：2026-06-12 你的版本：AutoCamTracker commit [f13c5d4](#) 朋友版本：SCP600/cartracking commit [a5c77af](#) 朋友 repo：<https://github.com/SCP600/cartracking>

1. 結論摘要

兩個版本的方向不是互相取代，而是互補。

你的版本比較像「可快速操作與展示的 V1.1 桌面原型」：Before / After 畫面、影片時間軸、播放速度、URL 影片輸入、Screen Region、YOLO model refresh、BoT-SORT / Deep OC-SORT 切換都已經集中在單一 Tkinter app 裡。優點是上手快、改 UI 快、測試使用者流程很直接。

朋友版本比較像「已經開始工程化的 tracking 系統骨架」：它把 app controller、pipeline worker、video source、detection、tracking、identity、framing、UI panel、recording 等拆成 package 模組，而且最重要的是已經把 [local_track_id](#) 與 [global_vehicle_id](#) 分開。這代表它更接近後續要處理「車輛短暫消失、鏡頭切換、track id 重置後仍記得選取車輛」的架構。

建議路線：保留你的 UI/輸入/播放體驗，逐步吸收朋友版本的 pipeline worker、FrameData、GlobalIdentityManager、ReacquireEngine、SceneCutDetector 與 RecognizedVehicleRegistry。不要直接整包替換，否則你現在已經能操作的 URL、timeline、Deep OC-SORT、比例修正和 V1.1 UI 會被打斷。

2. 功能比較

面向	你的版本 AutoCamTracker	朋友版本 SCP600/cartracking	判斷
輸入來源	webcam、video_file、screen_region、video_url	webcam、file、screen	你的版本多了 URL 影片，且影片播放控制較完整。
URL 影片	支援 YouTube / 網頁影片網址，透過 yt-dlp 解析成 stream URL	未看到 URL source	你的版本勝。
本機影片播放	支援 timeline seek、播放速度、skip late frames	基本 VideoFileSource，未看到 timeline UI	你的版本勝。
Screen Region	有全螢幕框選，重新框選會清掉上一個範圍	有 ScreenRegionSelector 與 preview	兩邊都有；你的版本近期修正了重新選取狀態。
YOLO 偵測	Ultralytics YOLO，model folder 掃描，可選 model	固定 yolo26n.pt，封裝於 YOLO26Detector	你的版本 model 操作更彈性；朋友版本封裝較乾淨。
Tracker	botsort、deepocsort	botsort、botsort_reid、bytetrack	兩邊方向不同；你的版本有 Deep OC-SORT adapter，朋友版本有 ReID BoT-SORT / ByteTrack 設定。
Target selection	Before 點 bbox、Auto Track	Current Detections list 選取	你的互動更直覺；朋友列表更適合 debug。
Identity	目前以 tracker track_id 為核心	明確分離 local_track_id 與 global_vehicle_id	朋友版本明顯勝。
Reacquire	TargetTracker 有 lost 狀態，但主要仍依 track id	ReacquireEngine 用顏色、尺寸、位置、confidence、local tracker match 評分	朋友版本勝。
Camera cut	未看到正式 scene cut detection	SceneCutDetector 用 HSV histogram 偵測切鏡，切鏡後 reset local tracker 但保留 global identity	朋友版本勝。
Before / After	Before + After 主視覺完整，已修正來源比例	Raw / Detection View + Cropped / Output View	你的版本更適合 demo；朋友版本更適合監控。

面向	你的版本 AutoCamTracker	朋友版本 SCP600/cartracking	判斷
Recognized vehicles	目前無獨立 recognized registry	有 RecognizedVehicleRegistry 與 Recognized tab	朋友版本勝。
Status / Debug	status line 顯示 FPS、source、speed、selected、crop	StatusPanel 使用 structured FrameData，含 inference/tracking/reframe time、lost/reacquire score	朋友版本勝。
Threading	Tkinter loop 內讀 frame / track / render	PipelineWorker thread + queue，UI polling	朋友版本勝，較不易卡 UI。
Recording / Evaluation	Record 目前 scaffold	有 recording/video_recorder.py 、 evaluation_logger.py	朋友版本準備度較高。
測試	有 self_test.py ，偏 runtime / dependency / webcam / pipeline	有 5 個 core smoke tests	朋友版本單元測試粒度較好；你的 self-test 對環境檢查較實用。

3. 模組架構差異

你的版本

目前主要集中在 [code/V1/](#)：

- [app.py](#)：Tkinter UI、控制列、Before / After 顯示、timeline、事件處理、主 loop。
- [video_detector.py](#)：來源開啟、YOLO model 載入、OpenCV / mss 讀 frame、BoT-SORT / Deep OC-SORT 追蹤。
- [detection_store.py](#)：保存 current detections、track history、candidate ranking。
- [target_tracker.py](#)：選取 track id、lost / failed 狀態。
- [reframer.py](#)：依 bbox 計算 crop window，輸出 After frame。
- [tracker_adapter.py](#)：Deep OC-SORT adapter。

優點：

- 檔案少，理解成本低。
- UI 與功能改動速度快。
- 適合當前 V1.1 demo 迭代。
- URL、timeline、播放速度與 Before / After 操作已經更接近真實使用流程。

限制：

- [app.py](#) 責任過重，UI、狀態、播放、pipeline 控制混在一起。
- inference / tracking 在 Tkinter loop 中執行，長時間 YOLO 可能造成 UI 卡頓。
- target identity 仍是 track id 層級，車輛離開畫面或切鏡後容易失去原身份。
- status 是文字組合，後續做 recording、evaluation、debug dashboard 會不夠結構化。

朋友版本

主要 package 是 [autocam_tracker/](#)：

- [app/](#)：AppController、PipelineWorker、AppConfig、SourceConfig。
- [video/](#)：VideoFileSource、WebcamSource、ScreenRegionSource。
- [detection/](#)：YOLO26Detector、VehicleDetection、thumbnail crop。

- [tracking/](#) : tracker config、target selector/state、simple tracker。
- [identity/](#) : [GlobalIdentityManager](#)、[ReacquireEngine](#)、[VehicleIdentity](#)。
- [data/](#) : [DetectionStore](#)、[RecognizedVehicleRegistry](#)。
- [framing/](#) : [CropController](#)、[FramingController](#)。
- [ui/](#) : main window、control panel、live view、vehicle list、recognized list、status panel。
- [recording/](#) : video recorder、evaluation logger。

優點：

- 模組邊界清楚，適合多人平行開發。
- [PipelineWorker](#) 把重運算移出 UI thread。
- [FrameData](#) 是完整的跨模組資料契約。
- [global_vehicle_id](#) 讓產品目標和 tracker id 分離，方向正確。
- Reacquire、scene cut、recognized registry 都已經有可測試的第一版。

限制：

- 功能面還沒有你的 V1.1 完整，例如 URL 影片、timeline seek、播放速度、Deep OC-SORT adapter。
- UI 比較偏工程 debug，demo 操作感不如你的 Before / After 控制列完整。
- [YOLO26Detector](#) model path 固定在 project root 的 [yolo26n.pt](#)，模型管理彈性較低。
- Reacquire 目前是簡易 HSV histogram 與幾何評分，不是正式 Vehicle ReID；在同色車、多車密集、光線變化大時可能誤認。
- scene cut 用全畫面 HSV histogram，遇到快速曝光變化、賽道大面積同色、轉播 overlay 變化時可能需要調 threshold。

4. 優劣分析

你的版本優勢

1. 使用者操作流程比較完整：可以選來源、貼 URL、播放影片、調速度、拖時間軸、點 bbox、看 Before / After。
2. 已支援 YouTube / 網路影片 URL，這對拿網路賽車影片快速測 demo 很重要。
3. 已支援 Deep OC-SORT adapter，具備和 Ultralytics BoT-SORT 之外的 tracker A/B 比較能力。
4. V1.1 修正後 Before / After 已維持來源比例，比較符合實際畫面檢查。
5. 單一 app 原型易於快速試功能，短期 demo 開發效率高。

你的版本風險

1. 如果繼續把功能塞進 [app.py](#)，後面會變成難測、難拆、難多人合作。
2. 目前 selection 依賴 tracker id，這正好是賽車轉播最容易失效的地方。
3. 沒有 structured [FrameData](#)，後續 debug、錄影、CSV log、UI panel 都會一直從字串或各物件狀態拼資料。
4. 沒有 background worker，YOLO 慢時 UI 容易卡。
5. Reacquire 與 camera cut 還沒形成正式模組。

朋友版本優勢

1. 架構更接近後續可維護產品，模組可以平行開發。

2. identity 設計方向正確，`selected_global_vehicle_id` 不應該跟著 local tracker id 消失。
3. 有基本 reacquire、scene cut、recognized registry，且 smoke tests 已覆蓋核心概念。
4. UI / pipeline 以 queue 解耦，長期穩定度較好。
5. Status / FrameData 對工程 debug 很有利。

朋友版本風險

1. 功能完整度不如你目前版本，尤其是 URL、timeline、播放控制與模型選擇。
2. UI demo 感較弱，較像 debug console。
3. Reacquire 目前還是 heuristic，不應過度相信其身份判斷。
4. 若直接合併，容易把你的可用 demo 流程打散。

5. 建議整合策略

建議 1：不要直接替換 UI，先抽 pipeline

保留你的 `app.py` 使用者操作方式，但建立新的 pipeline 層：

```
code/V1/app.py
-> AppController
-> PipelineWorker
-> VideoSource
-> Detector
-> Tracker
-> IdentityManager
-> Reframer
-> FrameData
```

這樣你的 UI 不需要馬上重寫，卻能先得到 queue、structured data、identity、scene cut 的好處。

建議 2：優先導入 `FrameData`

先定義一個你版本的 `FrameData`，包含：

- raw frame / detection frame / after frame
- detections
- selected local track id
- selected global vehicle id
- tracking status
- fps / inference time / tracking time / reframe time
- crop window / zoom / error
- lost frames / reacquire score

這會讓 status panel、錄影、debug log、PDF 技術報告都變簡單。

建議 3：把 target identity 從 `track_id` 升級成 global identity

目前 `TargetTracker` 可以改成兩層：

- local layer：目前 tracker 回傳的 `track_id`

- product layer：使用者真正選的 `global_vehicle_id`

最小可行改法：

1. 點選 bbox 時建立 `global_vehicle_id`。
2. 保存該車的 last bbox、last center、thumbnail、color histogram。
3. 若 local track id 消失，進入 `SearchingTarget`。
4. 用朋友版本 `ReacquireEngine` 類似的 scoring 找候選車。
5. 找回後把新的 local track id 綁回同一個 global id。

建議 4：導入 scene cut，但不要讓它直接決定身份

朋友版本 `SceneCutDetector` 可以先移植，但建議只做三件事：

- reset local tracker state
- 保留 selected global identity
- 進入 `SearchingTarget`

不要在切鏡後馬上用第一個相似候選當作同一台車，至少要連續幾幀確認。

建議 5：保留你的 URL / timeline / Deep OC-SORT

這三個是你版本目前的實用優勢：

- URL 讓你能快速拿 YouTube 或網路影片測試。
- timeline / speed 讓你能回放問題片段。
- Deep OC-SORT 可以作為 BoT-SORT 的替代比較。

朋友版本可以反向吸收你的 `video_url` source 與 timeline 控制。

建議 6：逐步拆 `app.py`

拆分順序建議：

1. `source_controller.py`：webcam / file / URL / screen source 選擇與開啟。
2. `pipeline_worker.py`：read frame、detect、track、identity、reframe。
3. `frame_data.py`：統一 UI 收到的資料。
4. `identity_manager.py`：global id、lost、searching、reacquire。
5. `ui_panels/`：Source、Tracking、Playback、View、Status、BeforeAfter。

這樣拆不會一次改爆，也能和朋友版本對齊。

6. 建議 Roadmap

Phase 1：穩定 V1.1 demo

- 保留目前 UI。
- 補上 URL 錯誤訊息優化：顯示解析失敗、影片不可用、網路失敗等不同原因。
- 補一個 source smoke test：direct mp4 URL / YouTube URL / local file。
- 補一個 display aspect test：不同來源尺寸都不變形。

Phase 2 : 導入 background pipeline

- 新增 `PipelineWorker` 與 `FrameData`。
- UI 只 poll 最新 frame data。
- Start / Stop / Pause / Seek 改為向 worker 發 command。
- 避免 YOLO inference 卡住 Tkinter mainloop。

Phase 3 : 導入 global identity

- 新增 `GlobalIdentityManager`。
- 使用者選取後建立 `global_vehicle_id`。
- lost 不再立刻清除 selection，而是進入 `SearchingTarget`。
- 導入 `ReacquireEngine` heuristic。
- UI 顯示 local id 與 global id。

Phase 4 : 加入 recognized vehicle list

- 從朋友版本移植 `RecognizedVehicleRegistry`。
- 在 UI 加一個可折疊或 tab 的 recognized list。
- 每台車顯示縮圖、local track aliases、global id、last seen、match score。

Phase 5 : 工程化與測試

- 保留你的 `self_test.py` 做環境檢查。
- 新增朋友版本那類小型 unit tests：

- global identity survives missing frames - global identity survives camera cut - crop output keeps source shape - URL source resolves YouTube stream - screen region reselection clears previous region

7. 最終建議

短期不要把你的 repo 改成朋友 repo 的完整目錄結構。你的版本現在比較能跑 demo，應該先維持它的操作體驗。

但中期要把朋友版本的核心設計吸收進來，尤其是：

1. `PipelineWorker + queue`
2. `FrameData`
3. `GlobalIdentityManager`
4. `ReacquireEngine`
5. `SceneCutDetector`
6. `RecognizedVehicleRegistry`

最佳方向是：你的版本當產品前台，朋友版本當工程骨架參考。把「使用者已經覺得好操作的 UI」留下，把「追蹤身份與 pipeline 的工程結構」升級。

8. 本次檢查依據

- 已檢查你的 repo : README.md、code/V1/app.py、video_detector.py、detection_store.py、target_tracker.py、reframer.py、tracker_adapter.py。
- 已檢查朋友 repo : docs/spec.md、app_controller.py、pipeline_worker.py、yolo26_detector.py、global_identity_manager.py、reacquire_engine.py、recognized_vehicle_registry.py、crop_controller.py、control_panel.py、main_window.py。
- 已執行朋友 repo smoke tests : PYTHONPATH=/private/tmp/SCP600-cartracking .venv/bin/python tests/test_core_smoke.py , 結果 5 tests OK。